Comparación: orquestador propio vs. LangGraph — Toxiro Taller

Oct 20, 2026 · @ToxiroApps

1. Contexto entre agentes

El objeto de caso que se va enriqueciendo ya es el estado de LangGraph. Cada agente ve el contexto acumulado completo, no solo la salida del anterior, y eso no cambia.

Lo que LangGraph agrega:

- **Contrato explícito:** el estado declara qué campos existen y de qué tipo son.
- **Actualizaciones parciales:** cada nodo devuelve solo lo que cambió y LangGraph hace la mezcla. Hoy cualquier agente puede modificar cualquier parte del caso.
- **Reductores:** por campo se define si el valor nuevo reemplaza o se acumula.

La separación de `TenantContext` respecto del caso es una decisión de arquitecto, y LangGraph tiene un lugar para ella. El estado guarda lo que cambia durante la ejecución (síntoma, hipótesis, presupuesto, evaluación). El contexto de ejecución guarda lo fijo (catálogo, márgenes, reglas del taller). LangGraph 1.x permite pasar ese contexto a los nodos sin meterlo en el estado; la API exacta se confirma con Desarrollo en LG-002.

Decisión: ¿dónde va `TenantContext`?

	En el estado	Como contexto de ejecución
Ventaja	Un solo concepto; cualquier nodo lo lee igual	El estado queda limpio, solo con lo que cambia
Desventaja	El checkpointer lo guarda en cada paso y aparece entero en cada traza	Un concepto más que aprender
Gobernanza	Mezcla datos del caso con configuración del cliente	Separa configuración del cliente de los datos de ejecución

Recomendación del CTO: contexto de ejecución. En el día 2 el catálogo será un dato gobernado en Unity Catalog; no tiene sentido copiarlo en cada checkpoint y cada traza.

2. Reintento

La mecánica es casi igual; lo que cambia es dónde vive el contador.

	Hoy	Con LangGraph
Contador	Variable interna del orquestador	Campo `intentos` en el estado, lo incrementa el Evaluador
Retroalimentación	Registrada en el caso y pasada como entrada al Presupuestador	Campo `retroalimentaciones`, que el Presupuestador lee del estado
Bucle	`while` o `for` dentro del orquestador	Borde condicional: el Evaluador rechaza y el grafo vuelve al Presupuestador
Máximo	Constante en el orquestador	`MAX_REINTENTOS = 2` en configuración, leído por la función de ruteo

Hoy, si el proceso se cae a mitad del reintento, el contador se pierde y ninguna traza muestra en qué intento ibas. En el estado queda guardado en cada checkpoint y visible en LangSmith.

Para la entrevista: *el control de flujo dejó de ser memoria del proceso y pasó a ser estado auditable.*

3. Escalada

Acá está el valor real de la migración, y ya está en la cola de tickets pendientes de Toxiro Taller.

Hoy la escalada es un marcador terminal. El orquestador devuelve el caso con `status: escalated` y un `escalation_package`. El supervisor lo ve en el panel, pero no puede continuar el flujo.

Con `interrupt` + checkpointer + `thread_id`:

1. El Evaluador escala y el grafo se pausa en el nodo de revisión humana. No termina.
2. El estado completo queda guardado bajo un `thread_id`, que es el caso.
3. Horas o días después el supervisor aprueba, corrige o cancela, y el grafo se retoma desde ese punto sin volver a ejecutar Diagnóstico ni Presupuestador.

Límites, para no venderlo de más:

- LangGraph da el mecanismo de pausa y retoma, no la integración con el panel ni con FastAPI. El endpoint que recibe la decisión del supervisor y retoma el `thread_id` se sigue escribiendo a mano.

- En producción el checkpointer debe ser durable (Postgres o similar), no en memoria ni SQLite local.

Historia para la entrevista: *En mi sistema la escalada era un marcador terminal y retomar el flujo estaba en mi cola de pendientes, porque hacerlo a mano implicaba serializar el estado, guardarlo y reconstruir el punto de ejecución. LangGraph lo resuelve de forma nativa con interrupt y checkpointer. Lo implementé en la migración y eso cambió la escalada de aviso a control humano real dentro del flujo.*

4. Lo que se pierde y cuándo elegir cada uno

Regla: orquestador propio si el flujo es lineal, corto y sin intervención humana; LangGraph cuando hay que pausar, retomar, auditar el control de flujo o ramificar. Toxiro Taller cae en el segundo caso por la escalada.

Lo que se pierde con LangGraph:

- **Control total:** hoy cada línea es conocida; LangGraph agrega una capa que hay que aprender a depurar.
- **Dependencia de un proveedor:** el sistema queda amarrado a su modelo de estado y su forma de persistir.
- **Sobrecarga en flujos simples:** una cadena lineal de 4 pasos sin pausas no necesita un grafo.

5. Decisiones pendientes

- [ ] ¿`TenantContext` va en el estado o como contexto de ejecución? Debe resolverse antes de LG-002; no bloquea LG-001.